Counters

Contemporary Digital Electronic Systems — Chapter 12

Muním Zabidi

Faculty of Electrical Engineering, Universiti Teknologi Malaysia

Table of Contents

1. Why This Chapter
2. Introduction
3. Asynchronous Counters
4. Synchronous Counters
5. Design of Synchronous Counters
6. Summary

Why This Chapter Matters

Counters are the simplest and most widely used sequential circuits: every digital clock, timer, frequency divider and sequence generator contains one. They are also the first circuits where the state is read as a number, a bridge from registers to the state-based thinking of Chapters 13–15.

After this chapter, you can:

1. Design asynchronous (ripple) counters of any modulus and explain their limitation.
2. Design synchronous binary up counters.
3. Design a modulo-*N* counter (*N* not a power of two), such as a decade counter, with the synchronous design procedure.
4. Use a counter as a frequency divider.

© Muním Zabidi

An Opening Question

A 50 MHz clock drives an LED that should blink once per second. What is the modulus of the counter between them?

We answer this with the frequency-division property of counters.

Table of Contents

1 Why This Chapter

2 Introduction

3 Asynchronous Counters

4 Synchronous Counters

5 Design of Synchronous Counters

6 Summary

What Is a Counter?

- A series of flip-flops whose outputs form a binary number; each clock pulse increments the count.
- Key difference, the clocking method:
  - Asynchronous (ripple): each stage's output clocks the next; stages change one after another.
  - Synchronous: one common clock; all outputs change together.
- Other classifications: modulus, count direction (up, down, up/down), edge polarity, flip-flop type.
- Modulus (mod) = number of states. n flip-flops give up to 2^n states: a 3-bit counter is mod-8.
- A counter with fewer than 2^n states is truncated: a decade (mod-10) counter counts 0000 to 1001.

Table of Contents

1 Why This Chapter

2 Introduction

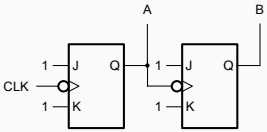
3 **Asynchronous Counters**

4 Synchronous Counters

5 Design of Synchronous Counters

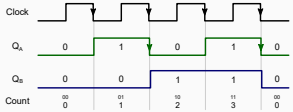
6 Summary

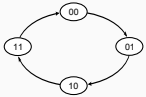
2-Bit Ripple Counter



Clock	Q_B	Q_A
Initial	0	0
1	0	1
2	1	0
3	1	1
4	0	0

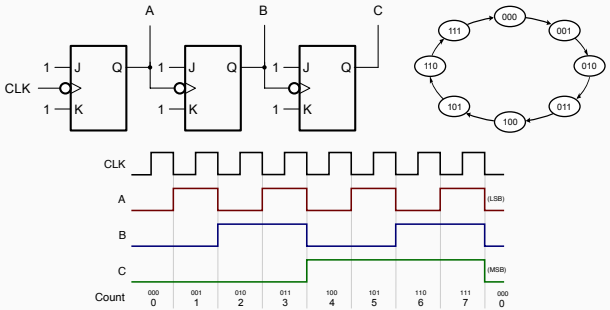
- The external clock drives only flip-flop A. Each falling edge toggles A.
- When A falls from 1 to 0, B toggles.





© Muním Zabidi

3-Bit Ripple Counter



- $2^3 = 8$ states: mod-8.
- Any flip-flop that can toggle works: T, JK with $J = K = 1$, or D with $\overline{Q}$ fed back to D.

Ripple Counters Divide Frequency

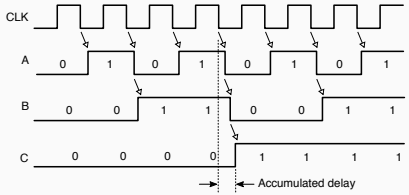
- Each flip-flop divides its clock by 2. With an 8 kHz clock, A, B and C output 4 kHz, 2 kHz and 1 kHz.
- A counter of modulus N divides the input frequency by N (if it cycles through its N states once per period).

An answer to the opening question

50 MHz to 1 Hz needs division by 50 000 000: a counter of modulus 50 000 000 ($\lceil \log_2 50\,000\,000 \rceil = 26$ flip-flops). A divide-by-N counter turns an f Hz clock into f/N Hz.

© Muním Zabidi

Limitation: Cumulative Propagation Delay



- Each stage waits for the previous one, so changes ripple.
- With delay t_p per stage, the output settles about nt_p after the clock edge:

$$T \geq nt_p, \qquad f_{\max} \leq \frac{1}{nt_p}$$

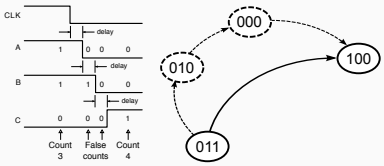
Simplified model: it ignores other timing requirements.

Example: 4-bit counter, $t_p = 12$ ns

$nt_p = 48$ ns, so $f_{\max} \leq 20.8$ MHz. Modulus 16: output at the last stage is about $20.8/16 \approx 1.3$ MHz.

© Muním Zabidi

False (Ghost) States



- Transition 011 $\rightarrow$ 100: A toggles (010), B toggles (000), C toggles (100).
- The intermediate states are temporary results of delay, not part of the sequence.
- They matter if other circuits read the outputs as a number while they are changing. Sample only after settling.

If a ripple counter is used only for frequency division, unused stages may be less restrictive; still check the flip-flop specifications.

Table of Contents

1 Why This Chapter

2 Introduction

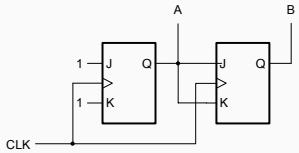
3 Asynchronous Counters

4 **Synchronous Counters**

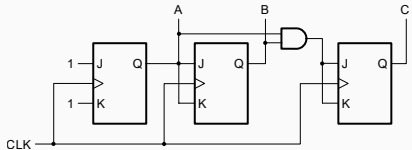
5 Design of Synchronous Counters

6 Summary

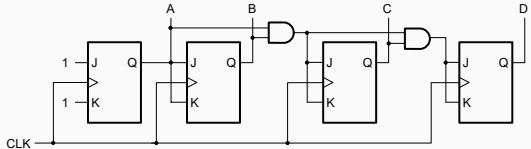
Synchronous Binary Counters



2-bit, mod-4



3-bit, mod-8

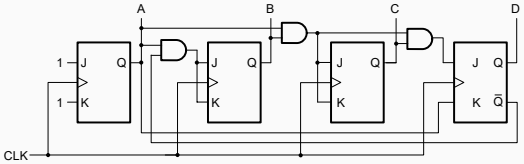


4-bit, mod-16

- One clock for every flip-flop: outputs change together, with no stage-to-stage delay.
- The LSB toggles on every pulse; each higher bit toggles only when all lower bits are 1. Combinational logic (AND gates) detects this.
- Costs extra logic, but runs faster.

© Munim Zabidi

Decade Counter and Speed



Mod-10: counts 0000 to 1001, then 0000

- Four flip-flops could hold 16 states, but this counter uses only ten.
- The longest path is one AND gate plus one flip-flop: $f_{\max} \le 1/(t_g + t_f)$: the delay does not accumulate stage by stage. The MSB AND gate widens with n and becomes a gate tree, so the delay grows only with $\log n$.

Example: $t_f = 12$ ns, $t_g = 5$ ns

$f_{\max} \le 1/17$ ns ≈ 58.8 MHz, about $2.8\times$ the ripple counter's 20.8 MHz.

Table of Contents

1 Why This Chapter

2 Introduction

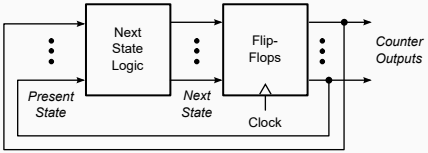
3 Asynchronous Counters

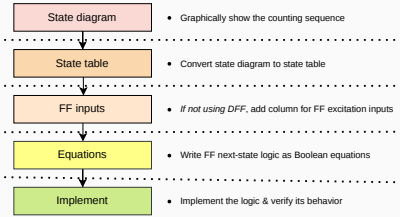
4 Synchronous Counters

5 **Design of Synchronous Counters**

6 Summary

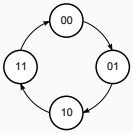
Counter Design Procedure





- A synchronous counter is flip-flops plus next-state logic: the counting sequence is set by that logic. This is finite-state-machine design (next chapter).
- Non-D flip-flops need an excitation table. For D flip-flops the input simply equals the desired next state.

Design with D Flip-Flops: 2-Bit Counter

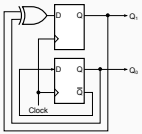


Q_1	Q_0	Q_1^+	Q_0^+	D_1	D_0
0	0	0	1	0	1
0	1	1	0	1	0
1	0	1	1	1	1
1	1	0	0	0	0

1. State diagram: arrows are unconditional.
2. State table: next state for each present state.
3. Flip-flop inputs: $D = Q^+$.
4. Equations by inspection:

$$D_1 = Q_1 \oplus Q_0 \qquad D_0 = \overline{Q_0}$$

5. Implement.



Example: Mod-5 Counter with D Flip-Flops

Q_2	Q_1	Q_0	Q_2^+	Q_1^+	Q_0^+
0	0	0	0	0	1
0	0	1	0	1	0
0	1	0	0	1	1
0	1	1	1	0	0
1	0	0	0	0	0
1	0	1	x	x	x
1	1	0	x	x	x
1	1	1	x	x	x

- Three unused states (101, 110, 111): their next states are don't cares.
- K-maps with the don't cares give:

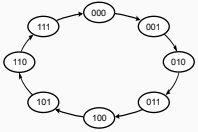
$$D_2 = Q_1 Q_0$$
$$D_1 = \overline{Q_2}\,\overline{Q_0} Q_1 + \overline{Q_2}\,\overline{Q_1} Q_0$$
$$D_0 = \overline{Q_2}\,\overline{Q_0}$$

Check self-correction

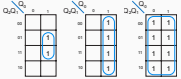
State 101 gives $D_2 D_1 D_0 = 000$: it returns to 000 next clock. Always verify this: don't-care simplification does not guarantee it.

Design with T Flip-Flops: Mod-8 Counter

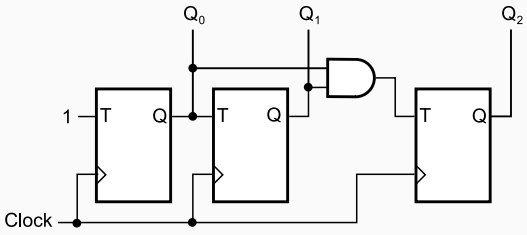
Q_2	Q_1	Q_0	Q_2^+	Q_1^+	Q_0^+	T_2	T_1	T_0
0	0	0	0	0	1	0	0	1
0	0	1	0	1	0	0	1	1
0	1	0	0	1	1	0	0	1
0	1	1	1	0	0	1	1	1
1	0	0	1	0	1	0	0	1
1	0	1	1	1	0	0	1	1
1	1	0	1	1	1	0	0	1
1	1	1	0	0	0	1	1	1



- A T input is 1 when the flip-flop must change.
- K-maps give $T_2 = Q_1Q_0$, $T_1 = Q_0$, $T_0 = 1$.



Mod-8 Counter Schematic and the General Rule



General rule for *n*-bit T-flip-flop up counters

$$T_i = Q_{i-1}Q_{i-2}\cdots Q_1Q_0, \quad T_0 = 1$$

Bit *i* toggles exactly when all lower bits are 1 (the ripple-carry condition).

A 4-bit counter needs $T_3 = Q_2Q_1Q_0$ with no fourth K-map. T flip-flops suit binary up counters: each T_i is a simple AND, unlike the D-flip-flop expressions.

© Muním Zabidi

Back to the Opening Question

Blinking an LED once per second

The counter divides 50 000 000 Hz down to 1 Hz, so its modulus is 50 000 000. That needs 26 flip-flops and is not a power of two, so the counter must be truncated (design it with the modulo-*N* procedure). A ripple version would be far too slow at 50 MHz, so a synchronous design is the sensible choice.

Table of Contents

1 Why This Chapter

2 Introduction

3 Asynchronous Counters

4 Synchronous Counters

5 Design of Synchronous Counters

6 **Summary**

Summary

- An asynchronous (ripple) counter clocks each stage from the previous one: simple, but delay accumulates and limits speed.
- A synchronous counter clocks every flip-flop together: faster, with more logic.
- Modulo-N counters (N not a power of two), such as a decade counter, are designed with the synchronous design procedure; check unused states for self-correction.
- A counter divides frequency by its modulus: divide-by-N turns f Hz into f/N Hz.

Looking ahead

Counters give an *ordered* sequence of states. When the next state depends on inputs, we need a *finite state machine*: Chapters 13–15.